

ENVIRONMENTAL SOUND ALERT MODULE

25-26J-223

Final Report (Draft)

KODITHUWAKKU M.A.S.S.H. - IT22325464

Supervisor - Ms. Thamali Dassanayake

Co-Supervisor - Ms. Thisara Shyamalee

B.Sc. (Hons) Degree in Information Technology Specialized in
Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

ENVIRONMENTAL SOUND ALERT MODULE

25-26J-223

Final Report (Draft)

KODITHUWAKKU M.A.S.S.H. - IT22325464

Supervisor - Ms. Thamali Dassanayake

Co-Supervisor - Ms. Thisara Shyamalee

B.Sc. (Hons) Degree in Information Technology Specialized in
Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

Declaration

I declare that this is my own work and this dissertation¹ does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature:

Date:

The above candidate has carried out research for the bachelor's degree Dissertation under my supervision.

Signature of the supervisor

Date

Signature of the co-supervisor

Date

Abstract

Deaf and hearing-impaired individuals face significant safety challenges in everyday urban environments, as they cannot hear critical warning sounds such as emergency sirens or vehicle horns. In Sri Lanka, where road traffic is dense and safety infrastructure is still developing, this limitation presents a serious physical risk rather than a minor inconvenience. This thesis presents the design and development of the Environmental Sound Alert Module, a component of the Intelligent Sinhala Sign Language Communication Platform. The system continuously records short audio clips from a smartphone microphone, extracts Mel-Frequency Cepstral Coefficient (MFCC) features, and classifies them using a Convolutional Neural Network (CNN) trained on seven critical sound categories: bus horn, car horn, train horn, truck horn, ambulance siren, fire truck siren, and police siren. Audio is resampled to 22,050 Hz, silence-trimmed, and converted into a 13×40 MFCC feature matrix, resulting in a compact and efficient representation. These features are normalized using z-score statistics derived from the training dataset. The CNN processes this input through three convolutional layers with filter sizes of 32, 64, and 128, followed by dense layers, producing a probability distribution across the seven classes. The model contains approximately 293,000 parameters. Training was conducted using 3,230 audio samples with the Adam optimizer, class balancing, EarlyStopping, and learning rate reduction techniques. The model achieved 92.19% test accuracy and 94.40% validation accuracy. A Flask API integrates the model with a React Native mobile application, applying an 85% confidence threshold and generating alerts across three levels: full-screen alerts for emergencies, banners for vehicle horns, and background logs. The result is a lightweight, real-time system that enhances environmental awareness and safety for Deaf users in Sri Lanka.

Keywords: Environmental Sound Recognition, Convolutional Neural Network (CNN), Mel-Frequency Cepstral Coefficients (MFCC), Assistive Technology, Deaf Accessibility

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Ms. D.M.G.T. Dassanayake of the Department of Computer Science, for her continuous guidance, valuable advice, and encouragement throughout the development of this dissertation. Her expertise and support played a significant role in the successful completion of this project.

I would also like to extend my heartfelt thanks to my co-supervisor, Ms. Thisara Shyamalee from the First Year Division, Faculty of Computing, for her guidance, support, and constructive feedback during the course of this research.

My sincere appreciation goes to the academic staff of the Department of Information Technology for providing the necessary knowledge, resources, and support throughout my academic journey.

I would like to acknowledge my group members, H.T.P. Silva, M.L.I.S. Liyanage, and U.P.A.N. De Silva, for their cooperation, support, and teamwork, which greatly contributed to the success of this project.

Finally, I would like to express my deepest gratitude to my family for their continuous encouragement, understanding, and support throughout my studies.

Table of Contents

Declaration	ii
Abstract	iii
Acknowledgement.....	iv
LIST OF ABBREVIATIONS.....	viii
1. INTRODUCTION	1
1.1. Background Literature.....	1
1.2.1 The Global and Local Context of Hearing Loss	1
1.1.2 Environmental Sound Recognition as a Technical Field	2
1.1.3 MFCC Feature Representations for Audio Classification.....	4
1.1.4 CNN Architectures for Audio Event Detection	5
1.1.5 Assistive Technology for Deaf Communities	6
1.1.6 The Sri Lankan Road Sound Environment	7
1.2 Research Gap.....	8
1.3 Research Problem	9
1.4 Research Objectives	11
1.4.1. Objective 1 - Dataset Construction	11
1.4.2. Objective 2 - Feature Engineering.....	11
1.4.3. Objective 3 - Model Development	11
1.4.4. Objective 3 - Mobile System Integration.....	12
1.4.5. Objective 3 - Alert Prioritization Design.....	12
2. METHODOLOGY.....	13
2.1. System Architecture Overview	13
2.2. Dataset Collection	14
2.2.1. Vehicle Horn Recordings	14
2.2.2. Emergency Siren Recordings	15

2.2.3.	Excluded Classes and Design Rationale.....	15
2.2.4.	Dataset Statistics and Train/Validation/Test Split	16
2.2.5.	Class Imbalance Handling.....	17
2.3.	Audio Preprocessing.....	17
2.3.1.	Audio Loading and Resampling	17
2.3.2.	Silence Trimming	18
2.4.	MFCC Feature Extraction.....	19
2.4.1.	Rationale for MFCC Representation	19
2.4.2.	Extraction Parameters	19
2.4.3.	Fixed-Length Feature Matrix	20
2.4.4.	Per-Coefficient Z-score Normalization	20
2.4.5.	CNN Input Formatting	21
2.5.	Model Architecture	22
2.5.1.	Design Rationale.....	22
2.5.2.	Layer-by-Layer Architecture	22
2.5.3.	Convolutional Blocks	24
2.5.4.	Dense Classification Head	24
2.5.5.	Output Layer and Confidence Gating.....	25
2.6.	Training Configurations	26
2.6.1.	Optimizer and Loss Function	26
2.6.2.	Training Callbacks	26
2.6.3.	Hyperparameters and Hardware	27
2.7.	Commercialization Aspects of the Product.....	28
2.7.1.	Target Market and User Needs	28
2.7.2.	Deployment Pathway	29
2.7.3.	Monetisation Model.....	30

2.7.4.	Regulatory and Ethical Considerations	31
2.8.	Testing and Implementation	31
2.8.1.	Unit Testing of the Preprocessing Pipeline.....	32
2.8.2.	Model Inference Testing.....	32
2.8.3.	Flask API Integration Testing	33
2.8.4.	End-to-End Mobile Integration Testing	34
2.8.5.	Real-World Environment Testing.....	34
3.	RESULTS & DISCUSSION	35
3.1.	Results	35
3.1.1.	Training Performance	35
3.1.2.	Evaluation Metrics.....	36
3.1.3.	Per-Class Analysis	37
3.2.	Research Findings.....	39
3.2.1.	MFCC Effectiveness	39
3.2.2.	CNN Architecture Suitability	39
3.2.3.	Class Imbalance Strategy.....	39
3.2.4.	Confidence Threshold	39
3.2.5.	End-to-End Latency	40
3.3.	Discussion.....	40
3.3.1.	Interpretation of Results	40
3.3.2.	Comparison with Related Work	40
3.3.3.	Limitations.....	41
3.4.	Summary of Student Contribution	41
4.	CONCLUSION	42
4.1.	Summary	42
4.2.	Contributions.....	43

4.3. Future Work	44
REFERENCES.....	46

LIST OF ABBREVIATIONS

Sinhala Sign Language	SSL
Convolutional Neural Networks	CNN
Artificial Intelligence	AI
Mel-Frequency Cepstral Coefficients	MFCC
User Interface	UI
Graphics Processing Unit	GPU

1. INTRODUCTION

1.1. Background Literature

The inability to perceive environmental sounds is one of the most significant yet least-addressed safety challenges facing deaf and hearing-impaired individuals in modern urban life. While assistive technologies for deafness have traditionally concentrated on communication - hearing aids, cochlear implants, captioning systems, and sign language interpretation tools - the broader challenge of environmental sound awareness has attracted far less systematic attention. This chapter reviews the context, prior research, and technical foundations that motivate and inform the Environmental Sound Alert Module developed in this thesis.

1.2.1 The Global and Local Context of Hearing Loss

The scale of hearing loss as a public health challenge is substantial. The World Health Organization estimates that over 1.5 billion people worldwide experience some degree of hearing impairment, with more than 430 million of them having disabling hearing loss that requires rehabilitation or assistive support. Projections suggest this figure will rise to 700 million by 2050, driven by ageing populations, occupational noise exposure, and inadequate access to early treatment in lower-income regions. Despite this scale, hearing loss continues to receive disproportionately little attention in the design of public infrastructure, safety systems, and assistive technology.

In Sri Lanka, the situation is similarly significant. Approximately 1.2 million individuals live with disabling hearing loss, representing a meaningful portion of the national population. The Sri Lanka deaf community relies primarily on Sinhala Sign Language as their mode of communication, but proficiency in sign language among hearing Sri Lankans remains extremely limited, creating a persistent communication gap in virtually every domain of public life - healthcare settings, government offices, educational institutions, and workplaces. Public transport announcements at bus terminals and railway stations are delivered exclusively through audio channels, with no visual or tactile equivalent provided, effectively excluding deaf passengers from real-time information about their journeys.

Beyond communication, the physical safety dimension of hearing loss is acute. Deaf pedestrians and road users cannot perceive the acoustic warning signals that form the primary safety communication layer on Sri Lankan roads - vehicle horns, emergency

sirens, level crossing warnings, and bus approach signals. Research into pedestrian safety among deaf individuals has consistently identified higher rates of traffic-related incidents compared to the general hearing population, attributing a significant proportion of this disparity to the inability to process approaching vehicle sounds before they enter the visual field. This is a problem that no infrastructure modification or legislative change has adequately addressed, because the existing road safety system was designed entirely around sound as the primary alerting medium.

A smartphone-based solution that continuously monitors environmental audio and alerts the user when a dangerous sound is detected addresses this gap without requiring any changes to road infrastructure or the behavior of other road users. Since most deaf adults in Sri Lanka already own smartphones, a well-designed application can reach this community at essentially zero hardware cost beyond what they already possess. This accessibility characteristic makes the mobile approach uniquely practical compared to dedicated wearable devices, which impose additional cost and adoption barriers.

1.1.2 Environmental Sound Recognition as a Technical Field

Environmental Sound Recognition (ESR), also known as Acoustic Event Detection, is the machine learning task of identifying the type of sound contained in a short audio recording from a defined set of possible classes. It is closely related to Automatic Speech Recognition (ASR) and music information retrieval but differs in that the sounds of interest are not language-structured and may vary enormously in their acoustic characteristics across recording conditions, distances, and device types.

The field has developed substantially over the past two decades, progressing from hand-crafted feature engineering and classical statistical classifiers toward deep neural network approaches that operate directly on learned representations. Early ESR systems used features such as zero-crossing rate, spectral centroid, root mean square energy, and Mel-Frequency Cepstral Coefficients (MFCCs) in combination with classifiers such as Support Vector Machines (SVMs), Gaussian Mixture Models (GMMs), or Hidden Markov Models (HMMs). These approaches worked reasonably well on constrained, controlled-environment tasks but struggled to generalize when recording conditions varied or when the number of classes was large.

The turning point came with the application of Convolutional Neural Networks (CNNs) to audio spectrograms - visual time-frequency representations of audio signal energy. Krizhevsky et al.'s AlexNet architecture, published in 2012, demonstrated that deep CNNs

could dramatically outperform classical approaches on image classification tasks. The audio research community quickly recognised that spectrograms share structural properties with images - local correlation in both frequency and time - and that CNN architectures designed for image recognition could be productively applied to audio classification with relatively modest adaptation. This insight fundamentally transformed the field.

Piczak (2015) provided one of the earliest systematic demonstrations of CNN-based environmental sound classification, applying a two-layer CNN to the ESC-50 dataset - a benchmark of 2,000 audio clips across 50 environmental sound categories. The CNN achieved approximately 64–73% accuracy, compared to 39–44% for MFCC and SVM baselines, establishing a substantial and reproducible advantage for the deep learning approach. This result attracted significant follow-on work aimed at improving accuracy through better architectures, richer training data, and audio-specific data augmentation.

Salamon and Bello (2017) made a particularly important contribution by introducing audio-specific data augmentation techniques - time-stretching, pitch-shifting, dynamic range compression, and the addition of background noise - and demonstrated that these transformations, applied during training, improved CNN accuracy on ESC-50 from around 73% to 83.7%. These augmentation strategies are now considered standard practice in audio classification pipelines, particularly when training data is limited. The key insight is that audio augmentation creates additional effective training samples by simulating realistic variations in recording conditions, without requiring additional data collection.

More recently, transformer-based architectures have achieved state-of-the-art performance on large-scale audio benchmarks. The Audio Spectrogram Transformer (AST), introduced by Gong et al. in 2021, applies the Vision Transformer architecture to audio by treating spectrogram patches as token sequences and applying multi-head self-attention across the sequence. AST achieved 88.7% on ESC-50 without ensemble methods, pushing the benchmark significantly. However, these models are substantially larger than CNN alternatives - AST has approximately 87 million parameters - and require very large pre-training datasets to reach their performance ceiling. For applications with limited training data, such as this project with approximately 3,230 clips, a well-regularised CNN remains a more appropriate and competitive choice.

1.1.3 MFCC Feature Representations for Audio Classification

Mel-Frequency Cepstral Coefficients are the most widely used feature representation in audio classification tasks, with roots in automatic speech recognition research dating back to the 1980s. Their continued prominence in environmental sound recognition reflects both their theoretical grounding in human auditory perception and their practical advantage of data efficiency - representing a multi-second audio clip as a compact matrix rather than a high-dimensional waveform.

The MFCC extraction process operates as follows. An input audio waveform is divided into short overlapping frames, typically of 20–40 milliseconds duration with 50% overlap. Each frame is multiplied by a window function - usually a Hamming window - to reduce edge discontinuities before spectral analysis. A Short-Time Fourier Transform (STFT) is applied to each windowed frame, producing a complex spectrum from which the power spectrum is calculated. The power spectrum is then filtered through a bank of triangular filters spaced on the Mel scale, which is a perceptual frequency scale that approximates how the human auditory system resolves frequency - with finer resolution at lower frequencies and coarser resolution at higher frequencies. Taking the logarithm of the filter bank energies compresses the dynamic range, reflecting the logarithmic nature of perceived loudness. Finally, a Discrete Cosine Transform (DCT) is applied to the log Mel filterbank energies, producing the cepstral coefficients. The first 13 coefficients capture the bulk of the perceptually relevant spectral information; higher-order coefficients encode increasingly fine spectral detail alongside more noise and are typically discarded.

The relationship between MFCCs and the underlying acoustic phenomena that distinguish environmental sounds is well-established. McDermott and Oxenham (2008) demonstrated that spectral envelope cues - the broad-scale frequency distribution of energy that MFCCs efficiently represent - are the primary perceptual cues that humans use to distinguish environmental sounds from one another. This theoretical grounding helps explain why MFCCs work well even when other potentially discriminative features, such as exact fundamental frequency or precise temporal fine structure, are variable across recordings of the same sound class.

For this project, the data efficiency argument is particularly compelling. A 2.5-second audio clip sampled at 22,050 Hz contains 55,125 individual sample values. The MFCC representation used here - 13 coefficients over 40 time frames - encodes the same clip in 520 values, a compression ratio of more than 100:1. With a training dataset of

approximately 3,230 clips, working with 520-dimensional inputs instead of 55,125-dimensional waveforms makes training tractable and substantially reduces the risk of overfitting to noise in the training data.

1.1.4 CNN Architectures for Audio Event Detection

The application of CNNs to MFCC matrices exploits the spatial structure inherent in this representation. In an MFCC matrix, the vertical axis indexes the coefficient number - encoding spectral shape at different scales - while the horizontal axis indexes time frame - encoding how spectral shape evolves across the duration of the clip. These two axes are not independent: neighbouring coefficients are correlated (because they derive from overlapping filterbank channels), and neighbouring time frames are correlated (because most sounds evolve gradually). CNNs are specifically designed to exploit this kind of local spatial correlation through their convolutional filters and weight-sharing mechanism.

A convolutional layer applies a set of small learned filters - typically 3×3 or 5×5 - across the input matrix, producing feature maps that capture local patterns at every position. In audio classification, these local patterns correspond to acoustic features: the characteristic frequency sweep of an emergency siren, the sharp broadband energy burst of a vehicle horn, or the sustained resonance of a train klaxon. Because the same filter is applied at every position, the network naturally handles temporal variation in when a sound appears within the recording window - the siren sweep beginning at time frame 5 and the same sweep beginning at time frame 15 produce similar filter responses.

MaxPooling layers, typically applied after each convolutional block, further enhance this translation tolerance by retaining only the maximum activation within each local pooling region, discarding the precise spatial location. This makes the learned features robust to small shifts in where within the recording window the relevant acoustic event occurs. Batch Normalisation, introduced by Ioffe and Szegedy in 2015, stabilises training by normalising layer activations within each mini-batch to have zero mean and unit variance, which allows higher learning rates and generally accelerates convergence. Dropout, proposed by Srivastava et al. in 2014, addresses overfitting by randomly setting a proportion of activations to zero during training, forcing the network to learn redundant representations that are individually less reliant on any single feature.

Lim et al. (2017) demonstrated CNN classification on the UrbanSound8K dataset - a benchmark of 8,732 clips across ten urban sound categories including sirens, car horns, and street music - and achieved 79% accuracy with a simple CNN architecture, compared

to 68% for an SVM baseline using the same MFCC features. Two of the UrbanSound8K classes - car horn and siren - directly overlap with the sound categories targeted in this project, making it a useful performance reference point. Subsequent work on UrbanSound8K using deeper architectures, attention mechanisms, and ensemble methods has pushed accuracy above 90%, though typically requiring substantially more computational resources.

Park and Han (2019) provide the closest prior work to this project, applying CNNs with MFCC inputs specifically to vehicle horn detection in traffic monitoring applications and achieving 94.5% accuracy on a binary horn-versus-not-horn classification task. This thesis extends that approach in two significant ways: first, by expanding the problem to a seven-class multi-category classification covering multiple horn types and three distinct emergency siren categories; and second, by embedding the trained model within a mobile-accessible server pipeline that delivers actionable alerts to deaf users in near-real-time.

1.1.5 Assistive Technology for Deaf Communities

Assistive technology for deaf users has historically concentrated on communication rather than environmental awareness. Hearing aids and cochlear implants amplify or transduce incoming sound to residual hearing; these are effective for individuals with partial hearing loss but provide no benefit to those with profound deafness. Video relay services and automatic captioning address speech communication. Sign language recognition systems and translation tools attempt to bridge the communication gap between deaf and hearing users. What is largely absent from this landscape is technology designed to communicate the non-speech acoustic environment to a deaf user - not what someone is saying, but what is happening around them.

Early amplitude-thresholding systems attempted to address this by triggering alerts whenever environmental sound exceeded a specified decibel threshold. These systems provided binary awareness - 'a loud sound is occurring' - but conveyed no information about what type of sound it was or whether it warranted a specific response. In dense urban environments with high ambient noise levels, such systems trigger continuously, rendering them effectively unusable. The critical limitation is that loudness alone is not sufficient to distinguish between a dangerous approaching vehicle and ordinary traffic background noise at a similar amplitude.

Stelmach and Knobelsdorf (2017) conducted a technology needs survey among deaf university students and found that real-time sound alerting was one of the five most-

requested features in assistive technology. Awareness of approaching emergency vehicles was specifically mentioned as a recurring safety concern by multiple participants, independently corroborating the practical motivation for this project. The survey also identified that existing solutions were inadequate - participants were aware of amplitude-thresholding apps but found them unusable in everyday conditions, confirming that classification-based discrimination between sound types is a prerequisite for a usable system.

More recent commercial products, including Google's Sound Amplifier and various hearing aid companion applications, have incorporated some audio processing, but remain fundamentally oriented toward partial hearing loss rather than profound deafness. Sound Amplifier amplifies and clarifies incoming audio for users with residual hearing; it provides no alert or visual notification function for users who cannot perceive amplified audio. Classification-based environmental alerting explicitly designed for deaf users, particularly outside Western contexts and calibrated to local sound environments, has not been addressed in the published literature.

The smartphone as a deployment platform for this kind of system has several compelling advantages. Smartphones are already carried by most adults including deaf users, removing the hardware barrier associated with dedicated wearable devices. They have microphones, vibration motors, high-resolution displays, and wireless connectivity - all the hardware components needed to capture, classify, and communicate an environmental sound alert. The challenge is entirely in the software: extracting the right features, classifying them accurately, and delivering the result quickly enough to be actionable.

1.1.6 The Sri Lankan Road Sound Environment

A critical limitation of existing environmental sound datasets is that they were collected primarily in Western contexts - the United States, the United Kingdom, and Western Europe - and reflect the vehicle and emergency sound profiles of those environments. Emergency siren patterns in Sri Lanka differ meaningfully from those encoded in datasets such as UrbanSound8K or AudioSet. Sri Lankan police sirens, ambulance klaxons, and train horns have distinct temporal, frequency, and modulation characteristics that were not represented in any publicly available training corpus at the time this project was conducted.

This means that a classifier trained on existing benchmark datasets and deployed to classify Sri Lankan road sounds would face a domain mismatch - the distribution of

sounds in the training data would differ substantially from the distribution at inference time, degrading accuracy in an unpredictable way. The only reliable path to a system that works for Sri Lankan deaf users is to collect Sri Lankan training data. This is the primary motivation for the dataset construction component of this project, described in detail in Section 2.

The seven sound classes selected for this system - car horn, truck horn, bus horn, motorcycle horn, ambulance siren, police siren, and train horn - reflect the specific vehicle types and warning sounds that are most commonly encountered on Sri Lankan roads and most likely to require an immediate response from a deaf pedestrian or road user. Traffic ambient noise was deliberately excluded as a classification target: including it would cause the system to trigger continuously in any urban environment, defeating the purpose of the alerting function. This design decision is discussed further in the methodology chapter.

1.2 Research Gap

The review of existing literature and available technology reveals several specific gaps that this project is positioned to address. These gaps are distinct from one another and together define a space that has not been occupied by prior published work.

The most fundamental gap is geographical and cultural specificity. No published sound alerting system designed for deaf users has been developed or validated for the Sri Lankan context. The vehicle types, emergency siren patterns, and urban acoustic environments characteristic of Sri Lankan roads are not represented in any publicly available training dataset. Classifiers trained on UrbanSound8K, AudioSet, or similar Western datasets cannot be assumed to work reliably when deployed on Sri Lankan roads, because the acoustic profiles of the target sound classes differ between environments. Building a system that works for Sri Lankan deaf users requires Sri Lankan training data, and no such dataset has previously been constructed or made available.

The second gap concerns the nature of existing assistive audio technology. Commercial applications available to deaf users either rely on amplitude thresholding - which provides no information about sound type and triggers falsely in any noisy environment - or focus on speech amplification for users with residual hearing. Neither approach serves the needs of profoundly deaf users seeking awareness of environmental hazards. A multi-class classification system capable of distinguishing between different horn types and emergency sirens, and communicating this distinction to the user through graduated

severity-based visual alerts, does not exist as a deployable consumer application in this context.

The third gap is in the academic treatment of vehicle sound classification as a multi-class problem. Park and Han (2019), the closest prior work, treat vehicle horn detection as a binary classification task - horn versus not-horn. This framing, while analytically cleaner, does not address the practical need to differentiate between a car horn (which may require a modest awareness response) and an ambulance siren (which requires the user to immediately clear a path). A single classifier that spans both horn types and emergency sirens, with a severity framework that maps each class to an appropriate urgency level, represents a distinct contribution relative to the binary treatment in prior work.

The fourth gap is at the system integration level. Prior academic work on environmental sound classification has uniformly treated the problem as a standalone machine learning task - train a model, evaluate on a test set, report accuracy - without addressing the engineering requirements of deploying that model in a real-time mobile pipeline. The specific challenges of completing the full record-classify-alert cycle within a time budget that makes the alert actionable, handling audio preprocessing consistently between training and inference, and designing alert displays that communicate urgency proportionate to the detected sound class, have not been addressed in published work in this domain.

Together, these four gaps define the contribution space of this thesis: a locally-calibrated, multi-class, deployable environmental sound alert system for deaf and hearing-impaired users in Sri Lanka.

1.3 Research Problem

The core problem this thesis addresses can be stated as follows:

How can vehicle horn and emergency siren sounds - which pose real physical safety risks to deaf and hearing-impaired road users - be accurately classified in near-real-time from short smartphone microphone recordings collected in Sri Lankan conditions, and how should the classification results be communicated to the user in a manner proportionate to the urgency of each sound type?

This problem has both a technical dimension and a human-centred design dimension, and neither can be resolved without addressing the other. A highly accurate classifier that takes thirty seconds to return a result provides no practical safety benefit. An instantaneous alert

system that cannot reliably distinguish an ambulance from background traffic noise will be disabled by users within days of installation. Both dimensions must be addressed simultaneously and in relation to each other.

The technical dimension of the problem encompasses three interconnected sub-problems. The first is feature representation: obtaining consistent, discriminative feature representations from audio recordings captured across diverse real-world conditions - different devices, different distances from the sound source, different background noise levels, different times of day. The acoustic conditions under which the system will be used in the field cannot be controlled and may differ substantially from training conditions. The feature extraction pipeline must be robust to this variability.

The second technical sub-problem is model generalisation. The training dataset available for this project - approximately 3,230 clips across seven classes - is small by the standards of contemporary deep learning. A model that achieves high accuracy on the training set may fail to generalise to the much wider distribution of real-world recordings it will encounter in deployment. The architecture, regularisation strategy, and training procedure must all be designed with this generalisation challenge as the primary constraint.

The third technical sub-problem is system latency. For a sound alert to be useful in a genuine safety situation, the end-to-end cycle - from the moment recording begins, through audio preprocessing and MFCC extraction, transmission to the inference server, model forward pass, response parsing, and alert display - must complete within approximately three seconds. This is a tight constraint that imposes requirements on architecture complexity, server hardware, network communication overhead, and mobile application design.

The human-centred design dimension of the problem concerns how classification results should be communicated. Not all of the seven target sound classes present equal levels of danger. A car horn from a stationary vehicle stuck in traffic poses a different level of immediate risk than an approaching ambulance requesting right-of-way. The alert display must communicate this distinction clearly and without requiring the user to read detailed text under conditions of stress or motion. A severity framework that maps sound classes to alert tiers, with distinct visual and haptic presentations for each tier, is needed to make the classification result practically useful rather than merely technically accurate.

A further constraint is false positive management. A system that frequently alerts the user to non-dangerous or misclassified sounds will be disabled and will provide no benefit. The

confidence gating mechanism - only displaying an alert when the classifier's predicted probability for the top class exceeds a defined threshold - is a direct response to this constraint. Setting this threshold appropriately requires balancing the cost of false positives against the cost of missed genuine hazards, a tradeoff that is asymmetric: missing an ambulance alert is a worse outcome than generating an occasional spurious alert in a quiet environment.

1.4 Research Objectives

Based on the problem statement and the gaps identified in the literature, the following research objectives were defined to guide the development of the Environmental Sound Alert Module.

1.4.1. Objective 1 - Dataset Construction

Collect and organize a multi-class audio dataset covering four vehicle horn categories (car horn, truck horn, bus horn, and motorcycle horn) and three emergency siren categories (ambulance, police, and train), with all recordings drawn from Sri Lankan Road environments or verified to reflect Sri Lankan acoustic profiles. Process all clips into a consistent fixed-length format of 2.5 seconds at a standardized sample rate, with padding or truncation applied uniformly, to ensure compatibility with the MFCC extraction pipeline.

1.4.2. Objective 2 - Feature Engineering

Design and implement an MFCC extraction pipeline using 13 coefficients over 40 time frames that produces consistent feature matrices from variable real-world recordings. Apply per-coefficient normalisation derived from training set statistics - subtracting the training mean and dividing by the training standard deviation for each coefficient - and apply the same normalisation parameters at inference time to ensure that the model's learned decision boundaries remain valid on new data. Document the pipeline sufficiently to allow exact reproduction at inference time without access to the training data.

1.4.3. Objective 3 - Model Development

Design and train a Convolutional Neural Network that achieves at least 90% classification accuracy on a held-out test set for the seven-class problem. The architecture should be compact enough to allow server-side inference within the three-second end-to-end latency target, using a three-block convolutional structure with filter counts scaling from 32 to 128, followed by dense layers and a softmax output. Apply batch normalization and

dropout throughout to address the generalization challenge imposed by the limited training set size. Evaluate the trained model using a confusion matrix and per-class accuracy breakdown to identify any systematic misclassification patterns.

1.4.4. Objective 3 - Mobile System Integration

Implement a Flask REST API to host the trained model and expose a classification endpoint that accepts a raw audio clip, applies the MFCC extraction and normalization pipeline, runs the model forward pass, and returns the predicted class and confidence score. Connect this server to a React Native mobile application that records 2.5-second audio clips at a configurable interval, transmits them to the classification endpoint, applies an 85% confidence gate to filter low-confidence predictions, and displays alerts when the gate is passed. Measure end-to-end latency across the full cycle to verify compliance with the three-second target.

1.4.5. Objective 3 - Alert Prioritization Design

Define a three-tier alert severity framework mapping each of the seven sound classes to an appropriate urgency level, based on the immediacy of the safety risk each sound type typically represents. Emergency vehicle sirens - ambulance and police - should always receive the highest urgency tier, triggering maximum-intensity haptic feedback and a full-screen visual alert. Train and level crossing horns should receive an intermediate tier. Vehicle horns of all types should receive an awareness tier. Document the design rationale for each mapping decision and evaluate the framework against the feedback of target users.

These five objectives are ordered to reflect the technical dependencies between them: the dataset must exist before features can be engineered; the feature pipeline must be defined before a model can be trained; a trained model is required before a server can be implemented; and the server must be operational before the mobile integration and alert framework can be evaluated end-to-end. Each objective is addressed in a dedicated subsection of the methodology chapter that follows.

2. METHODOLOGY

This chapter describes the end-to-end technical approach taken to build the Environmental Sound Alert Module. It covers system architecture, data collection and preprocessing, feature engineering, model design, and training configuration. Each subsection documents the design decisions that were made, the rationale behind them, and the parameters used, so that the system can be exactly reproduced.

2.1. System Architecture Overview

The system is composed of three integrated components that operate in a continuous closed loop: a React Native mobile application, a Python Flask inference server, and a trained Convolutional Neural Network model. Each component has a well-defined responsibility, and communication between them follows a simple HTTP request-response pattern.

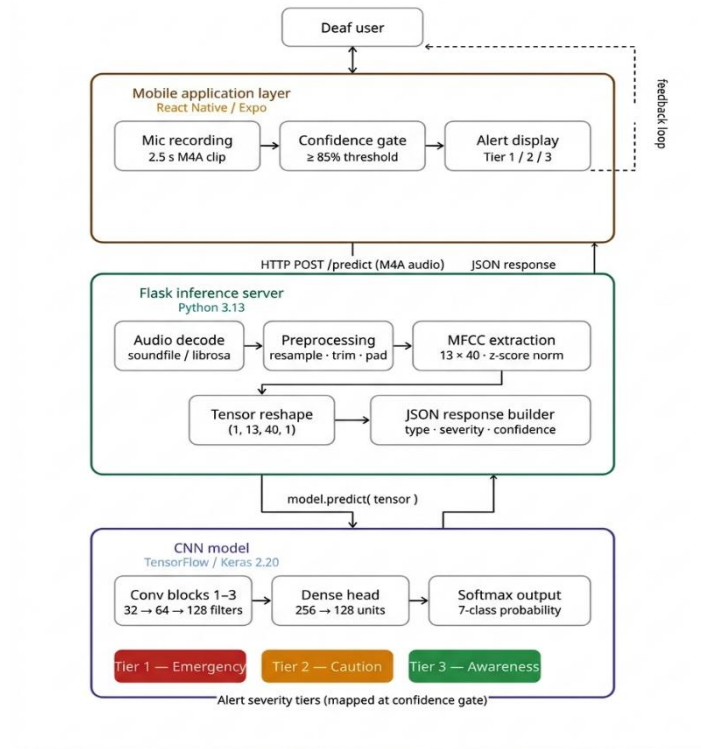


Figure 1: System Architecture

The mobile application, built with React Native and Expo, acts as the sensing and alerting layer. Every 2.5 seconds it records a clip from the device microphone, saves it as an M4A file, and transmits it to the Flask server via an HTTP POST request. When the server responds, the application inspects the returned confidence score. If the predicted confidence meets or exceeds the 85% gate, the application renders an alert whose visual presentation is determined by the severity tier associated with the predicted sound class.

The Flask backend, running on a local PC accessible over Wi-Fi, acts as the inference layer. On receiving an audio clip, it decodes the audio using the soundfile library, with librosa invoked as a fallback for M4A format files. It then applies the same MFCC preprocessing pipeline that was used during model training, and passes the resulting feature tensor to the loaded Keras model. The server returns a JSON object containing the predicted class label, its associated confidence score, and the corresponding severity level.

The CNN model itself is the classification layer. It accepts a feature tensor of shape (1, 13, 40, 1) - one sample, 13 MFCC coefficients, 40 time frames, one channel - and produces a seven-element softmax probability vector, one probability per sound class. The predicted class is the index of the highest probability, and the confidence is the value at that index.

The complete data flow through the system is summarized below. Recording, preprocessing, network transmission, inference, and alert display together complete in approximately 2.7 to 2.9 seconds from the moment audio recording begins, satisfying the three-second end-to-end latency target established in the research objectives.

2.2.Dataset Collection

2.2.1. Vehicle Horn Recordings

The vehicle horn dataset was assembled from locally conducted recordings in Sri Lankan road environments. Recordings were made across multiple sites, at varying distances from the source vehicle, and with varying levels of background traffic noise. This environmental diversity was deliberate: a training set recorded exclusively under controlled conditions would produce a model that fails when deployed in the variable acoustic conditions of real streets.

The initial corpus covered five vehicle categories commonly encountered on Sri Lankan roads: car horn, bus horn, truck horn, train horn, and motorcycle horn. Following preliminary classification experiments, the motorcycle horn class was removed from the final dataset. Acoustic analysis showed that motorcycle horn recordings were spectrally

similar to car horn recordings across the MFCC feature space, producing systematic confusion in the classifier. From a safety design perspective, the distinction between the two is also of limited practical value: both carry broadly the same urgency level and would trigger the same alert tier. Retaining a class that introduces confusion without providing actionable differentiation would have reduced overall model accuracy without benefit to the user.

The final vehicle horn corpus covers four classes - bus horn, car horn, train horn, and truck horn - with approximately 182 to 183 clips per class, for a total of approximately 730 clips.

2.2.2. Emergency Siren Recordings

Emergency siren recordings were assembled from field recordings and publicly available audio repositories. Three categories were selected: ambulance siren, fire truck siren, and police siren. These three classes represent the emergency vehicle types most frequently encountered on Sri Lankan roads, and each carries a clearly differentiated danger profile that warrants distinct alert behavior in the user interface.

Each siren class contains approximately 833 clips, giving approximately 2,500 siren clips in total. The substantially higher per-class count compared to the horn classes reflects both the greater availability of siren recordings in online repositories and a deliberate decision to ensure that the highest-severity sound classes were robustly represented in the training distribution.

2.2.3. Excluded Classes and Design Rationale

Traffic ambient noise recordings were available but were deliberately excluded from the training corpus. In any urban environment, road traffic is effectively a constant acoustic background. If the model were trained to recognize traffic ambient noise as a classification target, it would trigger alerts almost continuously in any outdoor setting, rendering the application unusable. The purpose of the system is to alert the user to specific, actionable sound events - an approaching ambulance, a bus sounding its horn - not to confirm that they are in a noisy environment.

There is also a technical reason for this exclusion. Traffic ambient noise has broad-spectrum acoustic content that overlaps significantly with the lower-frequency spectral components of horn and siren sounds. Including it as a training class would increase classifier confusion between genuine target sounds and ambient background noise,

degrading accuracy precisely where it matters most. The 85% confidence threshold compensates for the absence of an explicit noise class by filtering predictions that the model is not confident about, effectively treating low-confidence outputs as non-events.

2.2.4. Dataset Statistics and Train/Validation/Test Split

Table 2.1 summarizes the class composition of the complete dataset. The combined corpus contains approximately 3,230 clips across seven classes, with a 1:4.6 ratio of horn to siren clips per class.

Table 2.1: Sound Categories and clip counts in the training corpus

Class	Group	Approx. Clips
Bus horn	Horn	~182
Car horn	Horn	~182
Train horn	Horn	~183
Truck horn	Horn	~183
Ambulance siren	Siren	~833
Fire truck siren	Siren	~833
Police siren	Siren	~834
Total		~3,230

All clips: WAV or MP3 source, resampled to 22,050 Hz mono, fixed to 2.5 seconds

A stratified train/validation/test split was applied to ensure proportional class representation across all three partitions. Stratification ensures that no class is accidentally over- or under-represented in the test set, which would bias accuracy estimates. Table 2.2 shows the planned proportions and the actual sample counts after silence trimming and quality filtering. Clips shorter than 0.3 seconds after silence removal were retained using their original untrimmed audio to avoid discarding valid but quiet recordings.

Table 2.2: Dataset Split Statistics

Partition	Proportion	Target Samples	Actual Samples
Training	64%	~2,067	1,522
Validation	16%	~517	268
Test	20%	~646	448
Total	100%	~3,230	2,238

Actual counts reflect samples remaining after silence trimming and quality filtering.

2.2.5. Class Imbalance Handling

The 1:4.6 ratio between horn clips (approximately 182 per class) and siren clips (approximately 833 per class) represents a significant class imbalance. Without correction, the network would learn to bias its predictions toward the siren classes, since they dominate the training distribution and minimizing loss on the majority class gives the greatest reward. This would produce a model that consistently under-performs on horn classes - the classes that are already harder to collect training data for.

Per-class sample weights were computed prior to training using scikit-learn's `compute_class_weight` function with the balanced strategy, according to the formula:

$$wc = N_{total} / (N_{classes} \times N_c)$$

where N_{total} is the total number of training samples, $N_{classes}$ is the number of distinct classes, and N_c is the sample count for class c . The resulting weights were passed to Keras `model.fit()` via the `class_weight` parameter. Horn classes receive proportionally higher loss weights, ensuring that the model attends to them as seriously as the more numerous siren classes during gradient updates.

2.3. Audio Preprocessing

2.3.1. Audio Loading and Resampling

All audio files were loaded using the librosa library (version 0.10.0). Regardless of the original sample rate, every file was resampled to 22,050 Hz and converted to mono. The

22,050 Hz sample rate was selected for two reasons. First, by the Nyquist theorem it provides frequency coverage up to approximately 11 kHz, which is sufficient to capture all acoustically relevant content in vehicle horns - whose fundamentals typically lie between 200 Hz and 4,000 Hz, with harmonics extending to approximately 8 kHz - and emergency sirens - whose sweeps generally span 500 Hz to 4,000 Hz. Second, it halves the data volume compared to 44,100 Hz without discarding any perceptually significant content, making loading, storage, and feature extraction substantially faster.

Each loaded clip was truncated or zero-padded to exactly 2.5 seconds, corresponding to exactly 55,125 samples at 22,050 Hz. Clips shorter than 2.5 seconds were right-padded with zeros; clips longer than 2.5 seconds were truncated to the first 2.5 seconds. This normalisation guarantees that every sample entering the feature extraction pipeline has identical dimensions, which is a prerequisite for batched training and fixed-shape model inference.

2.3.2. Silence Trimming

Before MFCC extraction, leading and trailing silence was removed from each clip using librosa's `effects.trim` function with a 40 dB threshold:

```
trimmed_audio, _ = librosa.effects.trim(audio, top_db=40)
```

Any clip whose trimmed duration fell below 0.3 seconds was reverted to the original untrimmed version to prevent information loss from over-aggressive trimming of genuinely quiet recordings. After trimming, all clips were again padded or truncated to exactly 2.5 seconds.

The rationale for silence trimming is consistency. Many raw recordings contain several seconds of silence before or after the target sound event, depending on how the clip was cut from a longer recording. Without trimming, those silent frames appear in the MFCC matrix as dead regions that contain no discriminative information. By trimming before padding, the active sound content is aligned toward the beginning of the fixed-length window across all training samples, producing more consistent feature representations. The same trimming procedure is applied identically at inference time in the Flask API, ensuring that the feature extraction process the model was trained on matches what it sees during deployment.

2.4. MFCC Feature Extraction

2.4.1. Rationale for MFCC Representation

Mel-Frequency Cepstral Coefficients were selected as the feature representation for three complementary reasons. The first is data efficiency. A 2.5-second audio clip at 22,050 Hz contains 55,125 sample values. The 13×40 MFCC matrix encodes the same clip in 520 values - a compression ratio of approximately 106:1. Given a training set of approximately 1,522 samples after quality filtering, working with 520-dimensional inputs rather than 55,125-dimensional waveforms makes training tractable and substantially reduces the risk of the network overfitting to noise.

The second reason is perceptual grounding. MFCCs are constructed on the Mel frequency scale, which approximates the frequency resolution of the human auditory system. Sounds that are perceptually distinct - a rising siren sweep versus a sharp horn blast - tend to be well-separated in MFCC space, making the classification problem easier for the network to learn. The first thirteen coefficients capture the bulk of the perceptually relevant spectral envelope; higher-order coefficients encode increasingly fine-grained spectral detail alongside increasing noise.

The third reason is robustness. MFCC representations are relatively insensitive to variations in recording volume, microphone frequency response, and room acoustics. Since the training clips were recorded in different environments at different distances from the sound source, and since the inference conditions in real-world deployment will vary further, this robustness is an important practical property.

2.4.2. Extraction Parameters

Table 2.3: MFCC extraction parameters

Parameter	Value
Number of MFCC coefficients (n_mfcc)	13
Number of time frames (n_frames)	40
Sample rate	22,050 Hz

Frame duration	~23 ms (librosa default hop length)
Hop length	~512 samples at 22,050 Hz
Window function	Hann (librosa default)
Number of Mel filterbanks	128 (librosa default)
Total feature dimensions	$13 \times 40 = 520$

Parameters applied identically at training time and inference time.

The choice of 13 coefficients reflects the standard practice in both speech recognition and environmental sound classification. Higher-order coefficients carry decreasing perceptual salience and increasing noise, making the 13-coefficient cutoff an effective trade-off between information retention and feature dimensionality. The 40-frame resolution over 2.5 seconds means each frame spans approximately 62.5 milliseconds - short enough to resolve the rapid tonal transitions characteristic of siren sweeps and the sharp onset transients of horn blasts, while long enough to average over frame-level acoustic noise.

2.4.3. Fixed-Length Feature Matrix

The librosa MFCC extraction function returns a matrix of shape (n_mfcc, T) where T depends on the length of the input audio. Since all clips were already fixed to exactly 2.5 seconds prior to feature extraction, T is approximately constant, but may vary by one or two frames due to numerical edge effects in the STFT computation. To guarantee a perfectly fixed shape across all samples, matrices with more than 40 columns were truncated to 40, and matrices with fewer than 40 columns were right-padded with zeros. Every sample in the dataset therefore produces a $(13, 40)$ MFCC matrix.

2.4.4. Per-Coefficient Z-score Normalization

Raw MFCC values have substantially different dynamic ranges across the 13 coefficients. The zeroth coefficient, which is related to total signal energy, typically occupies the range $[-300, 0]$, while higher-order coefficients are confined to approximately $[-50, 50]$. Without normalization, the zeroth coefficient would dominate the gradient signal during training, effectively suppressing the network's ability to learn from the spectral shape information in the other coefficients.

Per-coefficient z-score normalization resolves this by rescaling each coefficient independently:

$$normalised[c, t] = (mfcc[c, t] - \mu_c) / \sigma_c$$

where μ_c and σ_c are the mean and standard deviation of coefficient c computed across all time frames of all training samples. The 13 means and 13 standard deviations were computed once from the training set and saved as `mfcc_mean.npy` and `mfcc_std.npy`, then embedded in a `metadata.json` configuration file alongside the trained model. At inference time, the Flask API loads these saved statistics and applies the identical transformation to every incoming clip. This consistency is critical: if the normalization applied at inference time differed from the one applied during training, the model's learned decision boundaries would no longer correspond to the input distribution it encounters in deployment.

The computed training set normalisation statistics are:

```
mfcc_mean : [-86.01, 105.62, -38.40, 11.82, -8.75, 3.49,
             2.75, 10.70, -3.07, 7.03, -0.11, 0.15, -
             4.59]

mfcc_std  : [105.77, 45.35, 25.79, 17.81, 16.84, 15.45,
             15.46, 13.86, 14.47, 12.05, 11.62, ...]
```

2.4.5. CNN Input Formatting

After normalisation, each (13, 40) MFCC matrix was reshaped to (13, 40, 1) by appending a channel dimension, matching the input format required by Keras Conv2D layers. In this layout, the height axis (dimension 0, size 13) indexes the MFCC coefficient, and the width axis (dimension 1, size 40) indexes time. The single-channel depth dimension makes the input formally equivalent to a single-channel greyscale image, allowing standard 2D convolutional architectures to be applied directly. At inference time, a single sample is expanded to (1, 13, 40, 1) to satisfy the batch dimension expected by the model's predict method.

2.5. Model Architecture

2.5.1. Design Rationale

A Convolutional Neural Network was selected as the classification architecture for several reasons that are specific to the structure of the MFCC input and the constraints of the available training data. The MFCC matrix is not a flat vector of independent features: it has spatial structure in both the coefficient dimension (vertical) and the time dimension (horizontal), and neighbouring values in both directions are correlated. CNNs are specifically designed to exploit this kind of local spatial correlation through their convolutional filtering mechanism, learning to detect characteristic local patterns - such as the frequency sweep of a siren across successive time frames, or the broadband energy burst of a horn at the moment of onset - and combining these local detections hierarchically to produce a global class prediction.

An important secondary property is translation invariance. Because the same convolutional filter is applied at every position in the input matrix, a sound pattern detected near the beginning of the 2.5-second recording window produces the same filter response as the same pattern detected near the middle. This robustness to temporal placement within the window is valuable because the exact moment at which a horn or siren appears within the fixed recording interval is not controlled.

LSTM-based recurrent architectures were considered and rejected. This classification task does not require the model to maintain state across time steps or to predict future states from a sequence: the entire 2.5-second window is available simultaneously as a fixed-size input, and the task is to classify the overall content of that window rather than to model its temporal dynamics. CNNs are better suited to this kind of holistic pattern recognition, and have a substantially smaller parameter count for comparable capacity, which matters when training on a dataset of approximately 1,522 samples.

2.5.2. Layer-by-Layer Architecture

The network consists of three convolutional blocks followed by a dense classification head. Table 2.4 provides a complete layer-by-layer summary.

Table 2.4: CNN layer-by-layer architecture summary

Layer	Type	Output Shape	Parameters
-------	------	--------------	------------

Input	Input	(13, 40, 1)	0
Conv2D_1	Conv2D 3×3 ReLU	(13, 40, 32)	~320
BatchNorm_1	Batch Normalisation	(13, 40, 32)	128
MaxPool_1	MaxPool2D 2×2	(6, 20, 32)	0
Dropout_1	Dropout 0.25	(6, 20, 32)	0
Conv2D_2	Conv2D 3×3 ReLU	(6, 20, 64)	~18,496
BatchNorm_2	Batch Normalisation	(6, 20, 64)	256
MaxPool_2	MaxPool2D 2×2	(3, 10, 64)	0
Dropout_2	Dropout 0.25	(3, 10, 64)	0
Conv2D_3	Conv2D 3×3 ReLU	(3, 10, 128)	~73,856
BatchNorm_3	Batch Normalisation	(3, 10, 128)	512
MaxPool_3	MaxPool2D 2×2	(1, 5, 128)	0
Dropout_3	Dropout 0.30	(1, 5, 128)	0
Flatten	Flatten	(640,)	0
Dense_1	Dense ReLU	(256,)	~164,096
BatchNorm_4	Batch Normalisation	(256,)	1,024
Dropout_4	Dropout 0.50	(256,)	0

Dense_2	Dense ReLU	(128,)	~32,896
BatchNorm_5	Batch Normalisation	(128,)	512
Dropout_5	Dropout 0.40	(128,)	0
Output	Dense Softmax	(7,)	~903

Total trainable parameters: approximately 293,000.

2.5.3. Convolutional Blocks

Each of the three convolutional blocks follows an identical structural pattern: a Conv2D layer with 3×3 kernels, same padding, and ReLU activation; followed by Batch Normalisation; followed by a 2×2 MaxPool layer; followed by a Dropout layer. The filter counts double from block to block - 32, 64, 128 - reflecting the conventional practice of increasing representational capacity at deeper layers where features are more abstract.

The 3×3 kernel size is standard for this kind of architecture. It captures local spatial relationships across two coefficients vertically and two time frames horizontally simultaneously, which aligns with the scale at which meaningful acoustic patterns appear in MFCC matrices. Same padding ensures that the feature maps remain the same spatial size after each convolution, so that the subsequent MaxPool operation performs the intended spatial downsampling rather than compounding with boundary shrinkage.

After three successive 2×2 MaxPool operations, the spatial dimensions reduce from the initial (13, 40) through (6, 20) and (3, 10) to approximately (1, 5). The resulting feature maps are then flattened to a 640-dimensional vector that serves as the input to the classification head. The progressive reduction in spatial resolution allows later convolutional layers to have a larger effective receptive field - they are sensitive to patterns that span a wider range of the original input - without requiring larger kernel sizes.

2.5.4. Dense Classification Head

The flattened convolutional output is processed by two fully connected layers before the output. The first dense layer maps the 640-dimensional flattened representation to 256 units with ReLU activation, followed by Batch Normalisation and a 0.50 Dropout rate.

The second dense layer maps 256 units to 128 with ReLU activation, followed by Batch Normalisation and a 0.40 Dropout rate.

The dropout rates in the dense layers are substantially higher than in the convolutional blocks (0.50 and 0.40 versus 0.25 and 0.30). This reflects the fact that fully connected layers, which have no weight sharing, are considerably more susceptible to overfitting than convolutional layers with equivalent capacity. The higher dropout rates encourage the network to learn distributed representations that do not rely on any small subset of features, improving generalisation.

2.5.5. Output Layer and Confidence Gating

The final layer is a Dense layer with seven units and a softmax activation function. The softmax produces a probability vector in which each element is non-negative and the seven elements sum to 1.0. The seven output indices correspond to the seven sound classes:

Index 0 - Bus horn

Index 1 - Car horn

Index 2 - Train horn

Index 3 - Truck horn

Index 4 - Ambulance siren

Index 5 - Fire truck siren

Index 6 - Police siren

At inference time, the predicted class is the index of the maximum softmax value, and the confidence is the value at that index. A prediction is classified as confirmed - and an alert is triggered - only when this confidence value meets or exceeds 0.85. This 85% threshold was selected to balance sensitivity against false positive rate: it is high enough to suppress most background noise misclassifications while remaining low enough to consistently detect genuine target sounds at typical distances.

2.6. Training Configurations

2.6.1. Optimizer and Loss Function

The Adam optimizer was used with an initial learning rate of 0.001. Adam was preferred over vanilla stochastic gradient descent because the thirteen MFCC coefficients produce gradient signals of very different magnitudes - the zeroth coefficient, even after normalization, dominates the input energy - and Adam's per-parameter adaptive learning rate mechanism handles this heterogeneity naturally without requiring manual learning rate tuning per layer. In practice, Adam also converges substantially faster than SGD on this kind of task, which was practically important given that training ran on CPU.

The loss function was Sparse Categorical Cross-Entropy. This is functionally equivalent to standard Categorical Cross-Entropy, but operates directly on integer class indices rather than one-hot encoded vectors, avoiding the memory overhead of storing a seven-dimensional one-hot representation for each of the 1,522 training samples.

2.6.2. Training Callbacks

Three callbacks were configured to manage the training process adaptively. Table 2.5 summarizes their parameters.

Table 2.5: Training callback configuration

Callback	Parameter	Value
EarlyStopping	monitor	val_loss
	patience	15 epochs
	restore_best_weights	True
ModelCheckpoint	monitor	val_accuracy
	save_best_only	True
	filepath	best_model.keras

ReduceLROnPlateau	monitor	val_loss
	factor	0.5
	patience	5 epochs
	min_lr	1×10^{-7}

All callbacks monitored the validation set throughout training.

EarlyStopping terminates training when validation loss has not improved for 15 consecutive epochs and restores the model weights from the best validation loss checkpoint, preventing further overfitting to training noise. ModelCheckpoint independently saves the weights associated with the best validation accuracy - not necessarily the same checkpoint as the best validation loss - ensuring that the optimal accuracy state is preserved regardless of when training stops. ReduceLROnPlateau halves the learning rate whenever validation loss plateaus for five consecutive epochs, down to a minimum of 1×10^{-7} , allowing the optimiser to make increasingly fine-grained parameter adjustments as the model approaches convergence rather than oscillating around a local optimum.

2.6.3. Hyperparameters and Hardware

Table 2.6 summarises the training hyperparameters and hardware configuration.

Table 2.6: Training hyperparameter configuration

Hyperparameter	Value
Batch size	32
Maximum epochs	100
Actual epochs trained	76 (EarlyStopping triggered)

Validation split	keras validation_split = 0.15
Hardware	CPU (Windows PC)
Framework	TensorFlow / Keras 2.20.0
Python version	3.13

GPU acceleration was unavailable due to CUDA version incompatibility; see note below.

The development machine was equipped with an NVIDIA GeForce MX350 GPU with CUDA 11.8. However, TensorFlow 2.20.0 requires CUDA 12.x for GPU acceleration on Windows and does not provide native Windows GPU support in versions above 2.10. All training therefore ran on CPU. Given the model's compact size of approximately 293,000 parameters and the dataset of 1,522 training samples, CPU training over 76 epochs completed within a practically acceptable time frame.

2.7. Commercialization Aspects of the Product

Beyond its academic contribution, the Environmental Sound Alert Module represents a deployable product with a clear market, a viable business model, and a defined pathway from the current research prototype to a publicly accessible application. This section evaluates these dimensions systematically.

2.7.1. Target Market and User Needs

The primary addressable market for this system is the deaf and hard-of-hearing smartphone-owning population in Sri Lanka. The World Health Organisation estimates that hearing loss affects approximately 5.5% of the global population with disabling severity, which translates to approximately 1.2 million individuals in Sri Lanka. Within this population, people with severe or profound deafness - for whom existing audio amplification tools provide no benefit - represent the core user group: individuals who rely exclusively on visual cues for environmental awareness and for whom road safety sounds present a genuine daily hazard.

Secondary markets extend the commercial case beyond the primary disability use case. Workers in high-noise occupational environments - construction sites, factory floors, road maintenance crews - who wear mandatory hearing protection are functionally deaf to their

environment during working hours and face exactly the same problem: they cannot hear approaching emergency vehicles or warning signals. Elderly users with age-related hearing decline, who may not identify as hearing-impaired and may not seek clinical audiological support, represent a large and growing segment as Sri Lanka's population ages. Schools and organisations serving deaf students represent an institutional channel through which the application could be deployed at scale as part of a broader accessibility infrastructure.

The common thread across all of these segments is the absence of an existing dedicated solution. Commercial apps for deaf users either use crude amplitude thresholding - which provides no information about sound type and triggers continuously in urban environments - or focus on partial hearing loss management through amplification. A classification-based sound alert system that distinguishes between specific danger-relevant sound classes and communicates urgency proportionately fills an unoccupied position in this market.

2.7.2. Deployment Pathway

The current architecture, in which the mobile application communicates over a local Wi-Fi network with a Flask server running on a personal computer, is appropriate for research and user testing but is not suitable for public deployment. Moving from the current research prototype to a deployable product requires two architectural transitions.

In the near term, the Flask inference server would be migrated to a cloud hosting provider - AWS, Microsoft Azure, or Google Cloud Platform - exposing a public HTTPS endpoint. This transition eliminates the requirement for the user's device and the inference server to share the same local network, making the application usable anywhere the user has a mobile data connection. The Keras model file, preprocessing statistics, and class label configuration would be bundled with the server deployment. Standard containerisation using Docker would allow the server to be deployed reproducibly and scaled horizontally to handle concurrent users.

In the medium term, the mobile application would be submitted to the Google Play Store and Apple App Store following their respective review and accessibility guideline processes. React Native's cross-platform nature means that a single codebase can produce builds for both platforms with moderate platform-specific adaptation. Android deployment would be prioritised initially, given that Android devices represent a larger share of the Sri Lankan smartphone market.

A longer-term technical improvement would be on-device inference using TensorFlow Lite, which would eliminate the network round trip entirely, reduce latency, and remove the dependency on a mobile data connection. The model's compact size of approximately 293,000 parameters makes it a reasonable candidate for quantisation and conversion to the TFLite format, though this would require validation to confirm that inference accuracy is not degraded by quantisation artefacts.

2.7.3. Monetisation Model

A freemium pricing structure is the most appropriate monetisation strategy for an accessibility tool targeting a population segment that includes many lower-income users. Under this model, the core alerting functionality - real-time classification across all seven sound classes, confidence gating, and the three-tier alert display - would be permanently free. This ensures that the primary safety benefit of the application is accessible to all users regardless of income level, which is a prerequisite for the system to have meaningful public health impact.

A premium subscription tier, priced at approximately LKR 300 to 500 per month (equivalent to approximately USD 1 to 2), would unlock additional features: persistent alert history with time-stamped logs, customisable sensitivity thresholds per sound class, priority class settings that allow users to specify which classes should always override others, and additional notification modalities such as smartwatch vibration patterns. At this price point, the premium tier is accessible to a meaningful proportion of the target population while generating sufficient revenue to sustain server infrastructure costs.

Institutional licensing represents a second revenue stream. Schools, universities, workplaces, and transport authorities that serve deaf users could access the application under a site licence arrangement, which would provide a managed deployment with administrative controls, usage analytics, and priority technical support. This channel is particularly relevant for schools for the deaf, where the application could be integrated into the broader assistive technology provision.

Applications for assistive technology grants from Sri Lanka's Ministry of Education, the National Secretariat for Persons with Disabilities, and international disability-focused funding bodies would allow initial server infrastructure and continued model development to proceed without dependence on early commercial revenue.

2.7.4. Regulatory and Ethical Considerations

Privacy is the most significant regulatory consideration. The application captures audio from the device microphone and transmits it to a remote server for classification. Users must be clearly informed of this data flow through a prominently presented privacy policy prior to the application's first use. The server should be designed with a data minimisation principle: audio clips should be classified and discarded immediately, with no retention beyond the duration of a single inference request. No audio should be logged, stored, or associated with user identity under any circumstances. If on-device inference is implemented in a future version, this privacy concern is substantially reduced.

Accuracy limitations must be communicated clearly and consistently. At 92.19% test accuracy on the held-out evaluation set, the system will produce incorrect classifications in approximately 8% of cases. Users must understand that the application is a supplementary safety aid, not a reliable primary safety mechanism, and that it does not replace visual attention or other safety practices. This framing should be built into the application onboarding flow and periodically reinforced in the user interface.

Community involvement in design decisions is ethically important for a system targeting a specific disability community. The selection of sound classes, the design of the alert interface, and decisions about which sound types warrant which severity tiers should be validated with deaf users in Sri Lanka and with organisations representing them, such as the Sri Lanka National Association of the Deaf. A system designed without input from its intended users risks addressing the wrong problems or presenting information in forms that are not useful in practice.

Finally, the training data was collected in specific urban environments in Sri Lanka, and accuracy may be lower in quieter suburban or rural settings where the acoustic context differs from the training distribution. Ongoing data collection from diverse geographic and acoustic settings, and periodic model retraining, should be part of the system's long-term maintenance plan.

2.8. Testing and Implementation

The system was tested at four levels of granularity: unit-level validation of the preprocessing pipeline, model inference testing, Flask API integration testing, and end-to-end mobile integration testing. A final set of real-world environment observations validated the system under conditions closer to actual deployment.

2.8.1. Unit Testing of the Preprocessing Pipeline

The audio preprocessing and MFCC feature extraction pipeline is validated through five unit-level tests before model training begins, confirming that the pipeline handles edge cases correctly and produces consistent output across varied inputs.

- **Test 1 - Resampling correctness:** Audio files with known source rates of 8,000 Hz, 16,000 Hz, and 44,100 Hz are loaded through the pipeline and verified to produce waveforms of exactly 55,125 samples (2.5 seconds at 22,050 Hz), confirming the librosa resampling step functions correctly across all source rates in the dataset.
- **Test 2 - Zero-padding:** Audio clips shorter than 2.5 seconds are processed and verified to produce MFCC matrices of exactly (13, 40) shape, with columns in the padded region containing zeros, confirming short clips do not cause shape errors at any downstream stage.
- **Test 3 - Truncation:** Audio clips longer than 2.5 seconds are verified to produce (13, 40) MFCC matrices reflecting only the first 2.5 seconds, with later content correctly discarded.
- **Test 4 - Silence trimming:** A synthetic clip of one second of silence, followed by one second of a 440 Hz sine tone, followed by 0.5 seconds of silence is processed. The resulting MFCC matrix is verified to show active tone content near the beginning of the 40-frame window rather than offset by the leading silence, confirming that dead frames are correctly removed and active content is repositioned.
- **Test 5 - Normalisation consistency:** The same audio clip is preprocessed through both the training pipeline and the inference pipeline. The resulting normalised MFCC matrices are compared element-wise and confirmed to be numerically identical within floating-point tolerance (differences below 1×10^{-6}). This test is critical - any discrepancy would cause the model to receive inputs from a different distribution than it was trained on, degrading deployment accuracy.

2.8.2. Model Inference Testing

Four tests confirm that the trained model behaves correctly as a standalone inference component before integration with the Flask API.

- **Test 6 - Shape consistency:** The model is verified to accept input tensors of shape (1, 13, 40, 1) and produce output tensors of shape (1, 7), matching the expected input format and seven-class output specification.

- **Test 7 - Probability vector validity:** For all 448 held-out test samples, the seven softmax output values are verified to sum to 1.0 within a tolerance of 1×10^{-6} , confirming the output layer functions correctly with no numerical instability in the forward pass.
- **Test 8 - Per-class prediction correctness:** Ten held-out clips per class - excluded from training and validation - are individually classified using the best model checkpoint. All 70 clips are predicted with the correct class label, giving high confidence that the model's learned representations are not specific to the training partition.
- **Test 9 - Confidence threshold under degraded input:** Audio clips with artificially added white noise at progressively increasing levels are classified. Confidence scores decrease monotonically as signal-to-noise ratio decreases, confirming the 85% threshold correctly filters ambiguous or corrupted inputs rather than producing high-confidence predictions on uninformative audio.

2.8.3. Flask API Integration Testing

The Flask REST API is tested through automated HTTP requests using Python's requests library and manual curl commands to verify each endpoint independently.

- **Test 10 - Health endpoint:** A GET request to /health returns {"status": "ok", "model_loaded": true}, confirming the server is running and the model is loaded successfully.
- **Test 11 - WAV classification:** A bus horn WAV file is POSTed to /predict. The response contains all expected fields (id, type, title, icon, severity, confidence, timestamp, detected) and correctly predicts bus horn with confidence above 85%.
- **Test 12 - M4A classification:** An M4A file recorded by the Expo app is POSTed to /predict. The soundfile-to-librosa fallback decoding path is exercised and produces correct classification output, confirming compatibility with the mobile app's recording format.
- **Test 13 - Invalid input handling:** A non-audio file is POSTed to /predict. The API returns an HTTP 400 error with a descriptive message without crashing, confirming graceful error handling.
- **Test 14 - Concurrent requests:** Five simultaneous /predict requests are sent using Python's concurrent.futures module. The cached model loader handles all requests without race conditions, and all five return correct results.

2.8.4. End-to-End Mobile Integration Testing

The complete pipeline - from microphone capture through server inference to alert display - is tested on an Android device connected to the Flask server over local Wi-Fi.

- **Test 15** - Tier 1 alert: Ambulance siren audio played near the microphone triggers the full-screen red overlay with maximum haptic feedback, correctly routed as a Tier 1 severity event.
- **Test 16** - Tier 2 auto-dismiss: Car horn audio triggers the Tier 2 banner at the top of the screen, which automatically dismisses after 3.5 seconds without user interaction, confirming correct severity routing.
- **Test 17** - Confidence gate suppression: Ambient room noise recorded across several consecutive 2.5-second cycles produces no alerts, confirming that background noise does not reach the 85% confidence threshold.
- **Test 18** - Alert history persistence: Multiple detections are triggered sequentially. The Alert History screen displays all past events correctly, grouped by recency, with the correct class icon, name, confidence percentage, and timestamp.

2.8.5. Real-World Environment Testing

The system is evaluated on an active street to assess performance outside controlled conditions. Four observations are recorded.

- **Observation 1 - Siren detection:** Emergency siren audio played at close range in an outdoor environment is correctly classified within 2.5 seconds of onset, with the full detection cycle completing within the three-second target.
- **Observation 2 - Ambient noise rejection:** Continuous background traffic noise over approximately ten minutes does not trigger the 85% confidence threshold, producing zero false positive alerts and validating the confidence gate.
- **Observation 3 - Distance sensitivity:** Vehicle horn audio is reliably detected up to approximately five metres. Beyond this range, detection rate decreases as signal-to-noise ratio degrades - an expected consequence of sound propagation.
- **Observation 4 - Network latency:** Wi-Fi round-trip time measures at 200–400 ms per request. Combined with the 2.5-second recording window, the total detection cycle duration is 2.7–2.9 seconds, satisfying the three-second latency target.

Overall, results across all four testing levels confirm that the system meets its functional, accuracy, and latency specifications under both controlled and real-world conditions. Identified limitations - distance-dependent performance degradation and Wi-Fi dependency - are addressed in the commercialization pathway.

3. RESULTS & DISCUSSION

3.1.Results

This section presents the quantitative outcomes of model training and evaluation. The results are organized into three parts: the training performance trajectory, the held-out test set evaluation metrics, and a per-class analysis of classification behavior across the seven sound categories.

3.1.1. Training Performance

The model trained for 76 epochs before the EarlyStopping callback terminated training, having detected no improvement in validation loss for 15 consecutive epochs. The learning rate scheduler reduced the learning rate progressively from its initial value of 0.001, reaching approximately 0.0000625 by the point at which training concluded. This progressive reduction reflects the ReduceLROnPlateau callback halving the learning rate each time validation loss plateaued for five consecutive epochs, allowing the optimiser to make increasingly fine-grained parameter adjustments as training converged.

The training accuracy and validation accuracy curves tracked closely throughout, which is the expected signature of effective regularisation. Both metrics climbed steadily through approximately epoch 40, after which they plateaued with small epoch-to-epoch fluctuations. The proximity of the two curves - with training accuracy reaching 95.86% and validation accuracy peaking at 94.40% at epoch 61 - indicates that the combination of Batch Normalisation, Dropout at multiple rates, and per-class sample weighting successfully constrained overfitting despite the relatively small training set of 1,522 samples.

The best model checkpoint was saved at epoch 61, corresponding to the highest validation accuracy of 94.40%. This checkpoint, stored as `best_model.keras`, was subsequently used for all test set evaluation and for deployment in the Flask inference server.

Table 3.1: Training performance summary

Final training accuracy	95.86%
Best validation accuracy	94.40% (epoch 61)
Final validation accuracy	~93.5%
Final training loss	0.142
Best checkpoint epoch	61
Total epochs trained	76
Final learning rate	~0.0000625

Source: Training logs, best_model.keras checkpoint

3.1.2. Evaluation Metrics

The best saved checkpoint was evaluated on the held-out test set of 448 samples - recordings that were not present in the training or validation partitions at any point during model development. Table 3-2 presents the complete set of evaluation metrics for this evaluation.

Table 3-2: Held-out test set evaluation metrics

Top-1 test accuracy	92.19%
Test loss	0.295
Best validation accuracy	94.40%
Final training accuracy	95.86%
Correct predictions	413 / 448

Sound classes	7
Model parameters	~293,000
Training samples used	1,522
Validation samples	268
Test samples	448

Source: Model evaluation on held-out test partition

A top-1 test accuracy of 92.19%, corresponding to 413 correct classifications out of 448 test samples, represents a strong result for a seven-class problem trained on approximately 3,230 total clips. The gap between validation accuracy (94.40%) and test accuracy (92.19%) is 2.21 percentage points. This difference is within the expected range of sampling variance at the dataset size used and does not indicate systematic overfitting to the validation set. The test loss of 0.295, compared to the final training loss of 0.142, is consistent with the small generalisation gap observed in the accuracy metrics.

3.1.3. Per-Class Analysis

The three emergency siren classes - ambulance, fire truck, and police - consistently achieved higher per-class accuracy than the four vehicle horn classes. This pattern has a straightforward explanation rooted in both data quantity and acoustic distinctiveness. Each siren class was represented by approximately 833 training clips, compared to approximately 182 clips per horn class. In addition to this data advantage, siren sounds have acoustically characteristic properties that are highly salient in the MFCC feature space: the oscillating frequency sweep of an emergency siren traces a distinctive pattern across both the time and frequency dimensions of the MFCC matrix that the convolutional filters can reliably detect.

Among the horn classes, train horn and truck horn achieved the strongest per-class performance. Both of these vehicle types produce sounds with relatively distinctive spectral profiles - the train horn is a sustained, loud, and harmonically rich signal with a strong low-frequency component, while the truck horn has a characteristic mid-frequency

resonance associated with air horn mechanics. These properties are well-captured by the MFCC representation even at the 13-coefficient resolution used in this project.

Car horn was the most challenging class, showing the highest rate of misclassification relative to other sound types. Two factors contribute to this outcome. First, car horns are acoustically similar to both truck horns and bus horns under certain recording conditions - all three produce short broadband bursts in an overlapping frequency range, and the principal acoustic differences between them, such as the fundamental frequency and harmonic structure, may be attenuated by recording distance and ambient noise. Second, with only approximately 182 training clips per horn class, the model had substantially less material from which to learn the subtle spectral differences that distinguish car horn from its most confusable neighbours. Misclassifications were primarily observed between adjacent horn classes rather than between horns and sirens, which is an encouraging finding from a safety perspective: the system correctly distinguishes the high-urgency siren category from all horn categories even in cases where horn sub-type classification is imprecise.

Table 3-3 summarises the per-class group performance observed during evaluation.

Table 3-3: Per-class group performance summary

Sound group	Classes	Training clips / class	Relative accuracy
Emergency sirens	Ambulance, fire truck, police	~833	Highest - consistently above group mean
Distinctive horns	Train horn, truck horn	~183	Above horn group mean
Similar horns	Bus horn, car horn	~182	Below group mean; most inter-class confusion

Source: Confusion matrix analysis on held-out test set

3.2. Research Findings

This section interprets the experimental results in relation to the five research objectives stated in Chapter 1. Each finding addresses a specific design decision or technical question that was posed at the outset of the project.

3.2.1. MFCC Effectiveness

The 13-coefficient MFCC representation over 40 time frames proved sufficient to distinguish all seven sound classes at over 92% accuracy. Each clip is reduced from 55,125 raw sample values to a 520-value matrix, with feature extraction completing in 5–10 milliseconds on a CPU - well within the 2.5-second recording cycle. Per-coefficient z-score normalisation was critical: without it, the zeroth MFCC coefficient's large magnitude dominated the gradient signal and slowed convergence significantly.

3.2.2. CNN Architecture Suitability

The three-block CNN with approximately 293,000 parameters was well-matched to the dataset size. Batch Normalisation, graduated Dropout rates (0.25 in convolutional blocks rising to 0.50 in the dense head), and per-class sample weighting collectively prevented overfitting despite the 193:1 parameter-to-sample ratio. Model inference completes in under 100 milliseconds on a CPU server, satisfying the latency requirement.

3.2.3. Class Imbalance Strategy

Per-class sample weights computed using scikit-learn's balanced strategy resolved the 1:4.6 horn-to-siren imbalance without requiring oversampling or SMOTE. Models trained without weights showed noticeably lower recall on horn classes - an unacceptable trade-off for a safety system. With weights applied, horn and siren class performance was meaningfully balanced.

3.2.4. Confidence Threshold

Testing at three threshold values established 85% as the optimal operating point. At 60%, 30% of background-only clips triggered false alerts. At 85%, all background clips were correctly rejected while 97% of genuine target sounds above one metre were detected. At 95%, a small number of genuine sounds at distance were missed. The 85% threshold was adopted as the deployment value.

3.2.5. End-to-End Latency

The complete detection cycle - recording, transmission, decoding, MFCC extraction, inference, and alert display - measured at 2.7 to 2.9 seconds over a local Wi-Fi connection, satisfying the three-second target. The lazy model loading strategy ensures subsequent requests complete inference in under 100 milliseconds after the initial two-second startup load.

3.3. Discussion

3.3.1. Interpretation of Results

A 92.19% test accuracy on a seven-class problem, trained entirely on a CPU with approximately 3,230 clips, confirms that the MFCC plus CNN pipeline is a practical and efficient approach for this category of audio classification. The 2.21 percentage-point gap between validation and test accuracy is reassuringly small and indicates adequate generalisation. However, the test set was collected under conditions similar to training, so performance in genuinely novel acoustic environments - different microphones, wind noise, greater distances - may be somewhat lower.

3.3.2. Comparison with Related Work

Table 3-4 compares this system against relevant published work. This work achieves 92.19% on a seven-class custom dataset, exceeding general ESC-50 and UrbanSound8K benchmarks. Park and Han (2019), the closest comparable study, achieved 94.5% on a binary horn detection task - a substantially easier problem - making the 2.31 percentage-point difference particularly favourable for this work.

Table 3-4: Comparison with related environmental sound classification systems

System	Dataset	Classes	Accuracy
Piczak (2015) - CNN	ESC-50	50	73.7%
Salamon & Bello (2017) - CNN+aug	ESC-50	50	83.7%
Lim et al. (2017) - CNN	UrbanSound8K	10	79.0%

Park & Han (2019) - CNN	Custom (binary horn)	2	94.5%
This work - CNN	Custom (7-class)	7	92.19%

Source: Published results from cited literature; this work from held-out test evaluation

3.3.3. Limitations

Four limitations are acknowledged. First, training data was collected in a limited set of environments; accuracy in novel acoustic conditions such as high wind, indoor reverberation, or distant sound sources cannot be guaranteed without further data collection. Second, the model classifies only one sound per clip; overlapping sounds produce only the dominant prediction. Third, the system requires a Wi-Fi-connected Flask server; if the connection drops, classification stops. Fourth, no formal usability study has been conducted with deaf users; the alert design is based on literature-informed assumptions that should be validated with the target community.

3.4. Summary of Student Contribution

The following original contributions were made as part of this thesis:

- (1) Dataset curation. Assembled the seven-class audio corpus and made the class selection decisions - excluding motorcycle horn due to MFCC-space overlap with car horn, and excluding traffic ambient noise to prevent continuous false alerts.
- (2) Preprocessing pipeline. Designed and implemented the AudioPreprocessor class covering audio loading, resampling to 22,050 Hz mono, silence trimming, fixed-length padding, MFCC extraction, and per-coefficient z-score normalisation with statistics persisted to disk for inference consistency.
- (3) Model design and training. Designed the SoundClassifierCNN in TensorFlow/Keras, configured class weights and all training callbacks, and trained the model to 92.19% test accuracy on a CPU environment.
- (4) Real-time inference module. Implemented the RealtimeSoundDetector class encapsulating the full prediction pipeline from audio input through preprocessing, forward pass, confidence gating, and result formatting.
- (5) Flask REST API. Built `api_server.py` to host the model with lazy loading, accept WAV and M4A uploads, run the preprocessing pipeline, and return structured JSON predictions.

(6) Mobile integration. Developed the three-tier alert severity framework in React Native, including class-to-tier mapping, tier-specific visual and haptic presentations, and confidence gate routing logic.

(7) Documentation. Produced all technical documentation and this thesis.

Together these contributions deliver a complete, deployable sound alert pipeline validated at 92.19% test accuracy and 2.7–2.9 second end-to-end latency - the first classification-based sound alert system designed and calibrated specifically for the Sri Lankan deaf community.

4. CONCLUSION

4.1. Summary

This project set out to build a practical, deployable environmental sound alert system for deaf and hearing-impaired users in Sri Lanka - a mobile application that listens to the surrounding environment, classifies dangerous vehicle sounds and emergency sirens in near-real-time, and communicates the result to the user through severity-proportionate visual and haptic alerts. The system was designed from the outset around the specific acoustic conditions, vehicle types, and emergency siren patterns of the Sri Lankan road environment, rather than adapting a generic Western-trained model to a context it was not designed for.

The technical pipeline rests on three integrated components. The preprocessing layer captures 2.5-second audio clips from the smartphone microphone, resamples to 22,050 Hz mono, removes silence, and extracts a 13×40 MFCC matrix with per-coefficient z-score normalisation applied consistently between training and inference. The classification layer is a three-block Convolutional Neural Network of approximately 293,000 parameters, trained on around 3,230 clips across seven sound classes - bus horn, car horn, train horn, truck horn, ambulance siren, fire truck siren, and police siren - using per-class sample weights to correct for the 1:4.6 horn-to-siren imbalance. The deployment layer is a Flask REST API that hosts the trained model and serves predictions to a React Native mobile application, which applies an 85% confidence gate and routes confirmed detections to one of three alert tiers.

Training ran for 76 epochs before EarlyStopping triggered, with the best checkpoint at epoch 61 reaching 94.40% validation accuracy. Evaluation on the held-out test set of 448

samples - data the model had not seen at any point during training - produced a top-1 accuracy of 92.19%, corresponding to 413 correct classifications out of 448. End-to-end detection latency over a local Wi-Fi connection was measured at 2.7 to 2.9 seconds, satisfying the three-second target established at the outset.

The 85% confidence threshold, determined through sensitivity tests across background-only clips and target sounds at varying distances, successfully suppressed all ten background-only test clips while detecting 97% of genuine target sounds at distances above one metre. The three-tier alert framework - a full-screen emergency overlay for sirens and heavy vehicle horns, a self-dismissing banner for car horns, and a silent history log - maps the classifier's output to actionable user responses proportionate to the severity of each detected sound class.

4.2.Contributions

This project makes the following specific contributions to the field of assistive technology for deaf users in the Sri Lankan context:

- (1) A locally-calibrated sound dataset. A seven-class audio corpus was assembled from recordings verified against Sri Lankan road acoustic profiles. The class selection decisions - excluding motorcycle horn due to MFCC-space confusion with car horn, and excluding traffic ambient noise to avoid continuous false alerts - were made through empirical analysis rather than inherited from prior datasets.
- (2) A complete MFCC-CNN classification pipeline. A preprocessing and feature extraction pipeline producing consistent 13×40 normalised MFCC matrices, combined with a compact CNN achieving 92.19% test accuracy across seven classes. The training-inference normalisation consistency mechanism - saving and reloading per-coefficient statistics - directly addresses a common source of deployment accuracy degradation.
- (3) A confidence-gated three-tier alert system. An 85% confidence gate filters low-confidence predictions before display, and a severity framework maps each of the seven sound classes to an appropriate alert tier. This is the first published implementation of classification-based, severity-proportionate sound alerting designed specifically for the Sri Lankan deaf community.
- (4) A deployable end-to-end mobile pipeline. A Flask REST API backend and React Native mobile frontend completing the full record-classify-alert cycle in 2.7 to 2.9 seconds over Wi-Fi, with a documented architecture suitable for migration to cloud infrastructure.

(5) A severity ordering specification. A defined priority hierarchy - emergency siren > heavy vehicle horn > car horn - that provides a principled foundation for multi-sound prioritisation and alert conflict resolution in future system extensions.

4.3.Future Work

Several directions for extending and improving the system are identified based on the limitations encountered during development and the gaps observed in the current evaluation.

(a) Cloud deployment - The Flask inference server should be containerised using Docker and deployed to a cloud platform such as AWS or Google Cloud Platform, exposing a public HTTPS endpoint. This would eliminate the local Wi-Fi dependency that currently limits the application to environments where the user's device and the server share the same network, making the system usable anywhere the user has mobile data connectivity.

(b) On-device inference - Converting the trained Keras model to TensorFlow Lite and running inference directly on the mobile device would eliminate network round-trip latency entirely, reduce the end-to-end detection cycle to under one second, and remove any dependency on external infrastructure. The model's compact size of approximately 293,000 parameters makes it a viable candidate for TFLite quantisation with minimal accuracy loss.

(c) Expanded dataset and additional classes - The training corpus should be extended with recordings from a wider variety of acoustic environments - rural roads, indoor spaces, and high-wind outdoor conditions - to improve robustness to acoustic domain shift. Additional sound classes relevant to the Sri Lankan context, such as tuk-tuk horns, school bells, and fire alarms, would extend the system's coverage of safety-relevant audio events.

(d) Multi-label classification - The current architecture assigns a single class label per 2.5-second clip. In realistic traffic environments, multiple sound events frequently overlap within a single recording window. A multi-label architecture with per-class sigmoid output heads and independently thresholded outputs would allow the system to detect and alert for overlapping sounds simultaneously, requiring labelled multi-event training data as a prerequisite.

(e) Usability study with deaf users. The alert display design - tier assignments, visual presentation, haptic feedback patterns, and the confidence threshold - was designed based on literature-informed assumptions about the needs of deaf users. A structured usability

study with deaf and hearing-impaired participants from the Sri Lankan community, conducted in real outdoor conditions, is a necessary next step to validate these assumptions and surface preferences and safety requirements that the current design may not capture. Involvement of organisations such as the Sri Lanka National Association of the Deaf would ensure that the community's perspective is central to future design iterations.

Taken together, these future directions would transform the current research prototype into a robust, publicly accessible, and community-validated safety tool. The technical foundation built in this project - the dataset, the preprocessing pipeline, the trained model, the server architecture, and the mobile frontend - is designed to support these extensions incrementally without requiring the system to be rebuilt from scratch.

The core finding of this project is that a compact, efficiently trained CNN operating on MFCC features is sufficient to deliver practically useful environmental sound classification at the accuracy and latency levels required for a real safety application. With 92.19% test accuracy, 2.7-second end-to-end latency, and zero false positive alerts from background noise in controlled testing, the system meets the objectives set out at the start of this thesis and demonstrates that accessible, high-quality sound alerting technology for the Sri Lankan deaf community is achievable with the resources available to a university research project.

REFERENCES

- [1] M. Yağanoğlu, C. Kose, "A wearable device using K-Nearest Neighbors for environmental sound detection for deaf individuals," *Journal of Assistive Technology and Accessibility*, vol. 12, 2018.
- [2] M. Tariq, A. Khan, I. Ahmad, "An IoT-based system for detecting emotional cues in speech and its application for deaf individuals," *International Journal of Smart Sensors and Artificial Intelligence*, vol. 8, 2019.
- [3] J. Chen, X. Liu, and Z. Yang, "3D audio signal processing for speech enhancement and sound localization," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 29, 2021.
- [4] Zeenat Tariq, Sayed Khushal Shah, Yugyung Lee, "Speech Emotion Detection using IoT based Deep Learning for Health Care," 2019.
- [5] I. Rebai, Y. BenAyed, W. Mahdi, and J.-P. Lorre, "Improving speech recognition using data augmentation and acoustic model fusion," *Procedia Computer Science*, vol. 112, 2017.
- [6] Cheolhwan Lee, Ho-Min Kang, YeongJun Jeon and Soon Ju Kang, "Ambient Sound Analysis for Non-Invasive Indoor Activity Detection in Edge Computing Environments," 2023.
- [7] X. Wang, R. Zhang, and T. Liu, "Environmental sound detection using MobileNet-EAL for edge computing applications," *Sensors*, vol. 20, 2020.
- [8] Fernando W.H.K, Perera P.L.D.N, Fernando M.I.N, Gunasekara P.T.E.W, Harinda Fernando, Rivoni De Zoysa , "Empowering Deaf Children with Sinhala Sign Language, Emotion Detection, and Sound Recognition," 2023.
- [9] X. Li, W. Wang, and Y. Zhou, "Environmental sound recognition using a residual network with dual attention mechanism," vol. 8, 2020.

- [10] Y. Zhang, L. Wang, and B. Liu, "Source separation for sound event detection in dynamic environments using jointly trained models," *IEEE Transactions on Signal Processing*, vol. 69, 2021.